

C & C++ Programming Cheat Sheet

This document provides a consolidated set of code examples from the C and C++ presentations, designed as a quick reference for students in COMPE496.

C Programming

Data Types

Basic Data Types

Plain Text

```
int age = 25;
float price = 19.99f;
char grade = 'A';
double pi = 3.14159265;

// Check size on platform
printf("Size of int: %zu\n", sizeof(int));
```

Fixed-Width Integers (<stdint.h>)

Plain Text

```
#include <stdint.h>

// Guaranteed sizes across platforms
int32_t exact_int = 100000;
uint8_t byte_val = 255;
int64_t big_num = 90000000000LL;
```

Operators

Arithmetic

Plain Text

```
int a = 10 / 3; // Result: 3 (Truncated)
int b = 10 % 3; // Result: 1 (Remainder)
float c = 10.0 / 3; // Result: 3.333...
```

Relational & Logical

Plain Text

```
// Common mistake: assignment vs. equality
if (x == 10) { ... } // Equality check
if (x = 10) { ... }  // ERROR: Assignment!

// Short-circuiting
if (ptr != NULL && *ptr > 0) {
    // *ptr is only accessed if ptr is not NULL
}
```

Increment/Decrement

Plain Text

```
y = x++;    // y gets the original value of x, then x increments
z = ++x;    // x increments, then z gets the new value of x
```

Control Flow

If-Else

Plain Text

```
if (score >= 90) {
    grade = 'A';
} else if (score >= 80) {
    grade = 'B';
} else {
    grade = 'F';
}
```

Switch

Plain Text

```
switch (day) {
    case 1:
        printf("Monday");
        break;
    case 2:
        printf("Tuesday");
        break;
    default:
        printf("Unknown");
}
```

For Loop

Plain Text

```
for (int i = 0; i < 10; i++) {  
    printf("%d ", i);  
}
```

While Loop

Plain Text

```
int count = 0;  
while (count < 5) {  
    printf("%d ", count);  
    count++;  
}
```

Do-While Loop

Plain Text

```
do {  
    // Executes at least once  
    scanf("%d", &num);  
} while (num != 0);
```

Arrays

1D Array

Plain Text

```
int numbers[5] = {10, 20, 30, 40, 50};  
numbers[0] = 15;    // Update first element  
int x = numbers[2];  // Read third element  
  
int len = sizeof(numbers) / sizeof(numbers[0]);
```

2D Array (Matrix)

Plain Text

```
int matrix[3][3] = {  
    {1, 0, 0},
```

```
    {0, 1, 0},
    {0, 0, 1}
};
int val = matrix[1][1]; // Access row 1, col 1
```

Pointers

Basics

Plain Text

```
int x = 10;
int *ptr = &x; // ptr points to the memory address of x

*ptr = 20;      // Dereference ptr to change the value of x to 20
```

Pointers and Arrays

Plain Text

```
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr; // An array name is a pointer to its first element

printf("%d\n", *p);      // Prints 10
printf("%d\n", *(p+1));  // Prints 20 (Pointer Arithmetic)

// Traverse array with a pointer
for (int *ptr = arr; ptr < arr + 5; ptr++) {
    printf("%d ", *ptr);
}
```

Functions

Structure

Plain Text

```
// 1. Prototype (Declaration)
int add(int a, int b);

int main() {
    // 3. Call
    int sum = add(5, 3);
    return 0;
}
```

```
// 2. Definition
int add(int a, int b) {
    return a + b;
}
```

Pass by Value vs. Pass by Reference

Plain Text

```
// Pass by Value (local copy is modified)
void inc_val(int x) {
    x++;
}

// Pass by Reference (original value is modified)
void inc_ref(int *x) {
    (*x)++;
}

int num = 5;
inc_val(num); // num is still 5
inc_ref(&num); // num is now 6
```

Structs

Plain Text

```
struct Student {
    char name[50];
    int id;
    float gpa;
};

// Initialization
struct Student s1 = {"John Doe", 12345, 3.75};

// Access with Dot (.)
s1.gpa = 3.80;

// Pointer to Struct
struct Student *ptr = &s1;

// Access with Arrow (->)
printf("ID: %d\n", ptr->id);
```

Dynamic Memory Management

Plain Text

```
#include <stdlib.h>

// Allocate array of 5 ints
int *arr = (int*)malloc(5 * sizeof(int));
if (arr == NULL) { /* Handle Error */ }

// Free the memory when done
free(arr);
arr = NULL; // Prevent dangling pointer

// calloc: allocates and initializes to zero
int *arr_zero = (int*)calloc(5, sizeof(int));

// realloc: resizes a memory block
arr = (int*)realloc(arr, 10 * sizeof(int));
```

Strings (`<string.h>`)

Plain Text

```
#include <string.h>

char buffer[10];

// Safe Input (prevents overflow)
fgets(buffer, sizeof(buffer), stdin);

// Safe Copy
char dest[10];
strncpy(dest, buffer, sizeof(dest) - 1);
dest[sizeof(dest) - 1] = '\0'; // Ensure null-termination

// Compare
if (strcmp(dest, "Hello") == 0) {
    printf("Match!\n");
}
```

File I/O (`<stdio.h>`)

Writing to a File

Plain Text

```
FILE *fp = fopen("output.txt", "w");
if (fp == NULL) { /* Handle Error */ }

fprintf(fp, "Score: %d\n", 100);
fputs("Game Over\n", fp);

fclose(fp);
```

Reading from a File

Plain Text

```
FILE *fp = fopen("input.txt", "r");
char buffer[100];
if (fp == NULL) { /* Handle Error */ }

while (fgets(buffer, sizeof(buffer), fp) != NULL) {
    printf("Read: %s", buffer);
}

fclose(fp);
```

C++ Programming

Basic Syntax (vs. C)

C Style

Plain Text

```
#include <stdio.h>

int main() {
    int success = 1; // 0 or 1 for boolean
    if (success) {
        printf("Status: %d\n", success);
    }
    return 0;
}
```

C++ Style

C++

```
#include <iostream>

int main() {
    bool success = true; // Native bool type
    if (success) {
        // Type-safe I/O streams
        std::cout << "Status: " << success << std::endl;
    }
    return 0;
}
```

Classes and Objects

C++

```
class Rectangle {
    int width, height; // private by default

public:
    void setValues(int w, int h) {
        width = w;
        height = h;
    }

    int area() {
        return width * height;
    }
}; // <-- Semicolon is mandatory

int main() {
    Rectangle rect; // Create an object (instance)
    rect.setValues(3, 4); // Call a method

    std::cout << "Area: " << rect.area();
    return 0;
}
```

Constructors and Destructors

C++

```
class Box {
public:
    // Default Constructor
    Box() {
```



```

        std::cout << "Box Created";
    }

    // Parameterized Constructor
    Box(int size) { /* ... */ }

    // Destructor
    ~Box() {
        std::cout << "Box Destroyed";
        // Resource cleanup happens here
    }
};

```

Inheritance

C++

```

class Animal { // Base Class
public:
    void eat() {
        std::cout << "Eating...";
    }
};

// Dog inherits from Animal (Derived Class)
class Dog : public Animal {
public:
    void bark() {
        std::cout << "Woof!";
    }
};

int main() {
    Dog myDog;
    myDog.eat(); // Inherited method
    myDog.bark(); // Own method
}

```

Polymorphism

Compile-Time (Overloading)

C++

```

class Printer {
public:

```

```
void print(int i) { std::cout << "Int: " << i; }  
void print(double f) { std::cout << "Float: " << f; }  
};
```

Runtime (Overriding)

C++

```
class Base {  
public:  
    virtual void show() { std::cout << "Base"; }  
};  
  
class Derived : public Base {  
public:  
    // 'override' keyword is good practice  
    void show() override { std::cout << "Derived"; }  
};  
  
int main() {  
    Base* ptr = new Derived();  
    ptr->show(); // Output: Derived  
    delete ptr;  
}
```

References vs. Pointers

Pointer

C++

```
int a = 10;  
int* ptr = &a; // ptr holds address of a  
*ptr = 30;     // a becomes 30  
ptr = nullptr; // Can be null
```

Reference

C++

```
int a = 10;  
int& ref = a; // ref is an alias for a  
ref = 30;     // a becomes 30  
// References cannot be null and cannot be reseated
```

Templates (Generic Programming)

Function Template

C++

```
template <typename T>
T add(T a, T b) {
    return a + b;
}

add<int>(5, 10);           // 15
add<double>(2.5, 3.1);    // 5.6
```

Class Template

C++

```
template <class T>
class Box {
    T value;
public:
    Box(T v) : value(v) {}
    T getValue() { return value; }
};

Box<int> intBox(100);
Box<std::string> strBox("Hello");
```

Standard Template Library (STL)

C++

```
#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    // 1. Container
    std::vector<int> nums = {4, 1, 3, 5, 2};

    // 2. Algorithm using Iterators
    std::sort(nums.begin(), nums.end());

    // 3. Iterating through the container
    for (int n : nums) {
```

```

        std::cout << n << " ";
    }
    // Output: 1 2 3 4 5
    return 0;
}

```

I/O Streams

Console I/O (<iostream>)

C++

```

int age;
std::cout << "Enter your age: ";
std::cin >> age;
std::cout << "You are " << age << " years old." << std::endl;

```

File I/O (<fstream>)

C++

```

#include <fstream>

// Writing to a file
std::ofstream outFile("data.txt");
if (outFile.is_open()) {
    outFile << "Saving data: " << 42;
    outFile.close();
}

// Reading from a file
std::ifstream inFile("data.txt");
std::string line;
if (inFile.is_open()) {
    while (getline(inFile, line)) {
        std::cout << line << std::endl;
    }
    inFile.close();
}

```

Modern C++ Features (C++11 and later)

Smart Pointers (<memory>)

C++

```
#include <memory>

void func() {
    // Manages memory automatically (RAII)
    auto ptr = std::make_unique<int>(10);

    // No 'delete' needed! Memory is freed when ptr goes out of scope.
}
```

Type Inference (auto)

C++

```
auto i = 42; // Compiler knows i is an int

std::vector<int> vec;
// Reduces verbosity, especially with complex types like iterators
for (auto it = vec.begin(); it != vec.end(); ++it) {
    // ...
}
```

Lambda Expressions

C++

```
std::vector<int> v = {1, 2, 3};

// Anonymous inline function: [capture](params){body}
std::for_each(v.begin(), v.end(), [](int n) {
    std::cout << n * n << " ";
});
// Output: 1 4 9
```